

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №10.

Тема: «Сохранение данных в файле с использованием потоков»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Создать пользовательский класс с минимальной функциональностью.
2. Написать функцию для создания объектов пользовательского класса (ввод исходной информации с клавиатуры) и сохранения их в потоке (файле).
3. Написать функцию для чтения и просмотра объектов из потока.
4. Написать функцию для удаления объектов из потока в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
5. Написать функцию для добавления объектов в поток в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
6. Написать функцию для изменения объектов в потоке в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
7. Для вызова функций в основной программе предусмотреть меню.

Создать класс `Pair` (пара чисел). Пара должна быть представлено двумя полями: типа `int` для первого числа и типа `double` для второго. Первое число при выводе на экран должно быть отделено от второго числа двоеточием.

Реализовать:

- операции сравнения (`<`, `>`).
- операция `--`, которая работает следующим образом: если форма операции префиксная, то уменьшается первое число, если форма операции постфиксная, то уменьшается второе число.

Задание:

- Удалить все записи равные заданному значению.
- Уменьшить все записи с заданным значением на число `L`.
- Добавить `K` записей после элемента с заданным значением

Анализ задачи:

1. Реализуем класс `Pair` с двумя полями
2. Перегружаем операции: сравнения (`<`, `>`) и декремент (`--`)
3. Реализуем работу с файлом/потокком данных:
 - a. Создание и сохранение объектов
 - b. Чтение и просмотр
 - c. Удаление записей по условию

- d. Добавление записей
 - e. Изменение записей
4. Делаем меню для управления всеми функциями

Код:

Pair.h

```
#pragma once
#ifndef PAIR_H
#define PAIR_H

#include <iostream>
#include <fstream>

using namespace std;

/**
 * Класс Pair представляет пару чисел: целое и вещественное.
 * Реализует перегрузку операций сравнения и декремента.
 * Формат вывода: "целое : вещественное"
 */
class Pair {
private:
    int first;      // первое число (целое)
    double second;  // второе число (вещественное)

public:
    // Конструкторы
    Pair() : first(0), second(0.0) {}
    Pair(int f, double s) : first(f), second(s) {}

    // Геттеры для доступа к полям
    int getFirst() const { return first; }
    double getSecond() const { return second; }

    // === Перегрузка операций сравнения ===
    // Сравнение производится по сумме значений полей
    bool operator<(const Pair& other) const {
```

```

        return (first + second) < (other.first + other.second);
    }

    bool operator>(const Pair& other) const {
        return (first + second) > (other.first + other.second);
    }

    bool operator==(const Pair& other) const {
        return first == other.first && second == other.second;
    }

    bool operator!=(const Pair& other) const {
        return !(*this == other);
    }

    // === Перегрузка операции декремента (--) ===
    // Префиксная форма: уменьшает первое число (first)
    Pair& operator--();

    // Постфиксная форма: уменьшает второе число (second)
    Pair operator--(int);

    // Уменьшение пары на значение (вспомогательный метод)
    void subtract(double value);

    // === Перегрузка операций ввода/вывода ===
    friend ostream& operator<<(ostream& out, const Pair& p);
    friend istream& operator>>(istream& in, Pair& p);
    friend fstream& operator<<(fstream& out, const Pair& p);
    friend fstream& operator>>(fstream& in, Pair& p);
};

// Реализация префиксного декремента: --pair уменьшает first
inline Pair& Pair::operator--() {
    --first;
    return *this;
}

```

```

// Реализация постфиксного декремента: pair-- уменьшает second
// Возвращает копию объекта до изменения (стандартное поведение постфикса)
inline Pair Pair::operator--(int) {
    Pair temp = *this; // сохраняем текущее состояние
    --second;           // изменяем second
    return temp;        // возвращаем старое значение
}

// Метод subtract: уменьшает сумму полей на value, пересчитывает поля
// Если результат отрицательный – обнуляем (защита от некорректных данных)
inline void Pair::subtract(double value) {
    double total = first + second - value;
    if (total < 0) total = 0;
    first = static_cast<int>(total); // целая часть -> first
    second = total - first;          // дробная часть -> second
}

// Вывод в поток: формат "first : second"
inline ostream& operator<<(ostream& out, const Pair& p) {
    out << p.first << " : " << p.second << '\n';
    return out;
}

// Ввод из потока: сначала целое, затем вещественное
inline istream& operator>>(istream& in, Pair& p) {
    in >> p.first >> p.second;
    return in;
}

// Запись в файловый поток (формат: два числа с переносом строки)
inline fstream& operator<<(fstream& out, const Pair& p) {
    out << p.first << '\n' << p.second << '\n';
    return out;
}

// Чтение из файлового потока
inline fstream& operator>>(fstream& in, Pair& p) {
    in >> p.first >> p.second;

```

```
    return in;
}
```

```
#endif
```

File_operations.h

```
#pragma once
```

```
#ifndef FILE_OPERATIONS_H
```

```
#define FILE_OPERATIONS_H
```

```
#include "pair.h"
```

```
#include <fstream>
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdio>
```

```
using namespace std;
```

```
// Создаёт новый файл и записывает в него указанное количество пар.
```

```
int createFile(const char* filename) {
```

```
    // Открываем файл на запись с флагом trunc (очистить, если существует)
```

```
    fstream file(filename, ios::out | ios::trunc);
```

```
    if (!file) return -1;
```

```
    int count;
```

```
    cout << "Enter the number of entries: ";
```

```
    cin >> count;
```

```
    Pair pair;
```

```
    for (int i = 0; i < count; ++i) {
```

```
        cout << "Record #" << (i + 1) << " (integer real): ";
```

```
        cin >> pair;
```

```
        file << pair; // используем перегруженный оператор <<
```

```
    }
```

```
    file.close();
```

```
    return count;
```

```
}
```

```

// Выводит содержимое файла на экран.
int displayFile(const char* filename) {
    fstream file(filename, ios::in);
    if (!file) return -1;

    Pair pair;
    int count = 0;
    cout << "\n--- File Contents ---\n";
    // Читаем до конца файла, используя перегруженный оператор >>
    while (file >> pair) {
        cout << pair; // используем перегруженный оператор <<
        ++count;
    }
    cout << "---- Total records: " << count << " ----\n";
    file.close();
    return count;
}

// Удаляет из файла все записи, равные заданному значению.
int deleteMatchingRecords(const char* filename, const Pair& target) {
    fstream input(filename, ios::in);
    fstream temp("temp.dat", ios::out | ios::trunc);
    if (!input || !temp) return -1;

    Pair pair;
    int deletedCount = 0;

    while (input >> pair) {
        // Если запись НЕ равна искомой – записываем во временный файл
        if (pair == target) {
            ++deletedCount; // считаем удалённые
        }
        else {
            temp << pair;
        }
    }

    input.close();

```

```

temp.close();

// Заменяем исходный файл временным
remove(filename);
rename("temp.dat", filename);
return deletedCount;
}

// Уменьшает все записи, равные target, на значение decrement.
int decreaseMatchingRecords(const char* filename, const Pair& target, double decrement)
{
    fstream input(filename, ios::in);
    fstream temp("temp.dat", ios::out | ios::trunc);
    if (!input || !temp) return -1;

    Pair pair;
    int changedCount = 0;

    while (input >> pair) {
        if (pair == target) {
            pair.subtract(decrement); // вызов метода класса
            ++changedCount;
        }
        temp << pair; // записываем (изменённую или нет)
    }

    input.close();
    temp.close();
    remove(filename);
    rename("temp.dat", filename);
    return changedCount;
}

// Добавляет countToAdd новых записей после первой записи, равной target.
int insertRecordsAfter(const char* filename, const Pair& target, int countToAdd) {
    fstream input(filename, ios::in);
    if (!input) return -1;

```

```

// Считываем все записи в вектор для удобной вставки
vector<Pair> records;
Pair pair;

while (input >> pair) {
    records.push_back(pair);
}
input.close();

// Поиск позиции для вставки (после найденного элемента)
auto it = records.begin();
bool found = false;

while (it != records.end()) {
    if (*it == target) {
        ++it; // сдвигаемся на позицию ПОСЛЕ найденного
        found = true;
        break;
    }
    ++it;
}

if (!found) {
    cout << "No entry found to insert!\n";
    return 0;
}

// Вставка новых записей в вектор
for (int i = 0; i < countToAdd; ++i) {
    cout << "New entry #" << (i + 1) << " (integer real): ";
    cin >> pair;
    it = records.insert(it, pair); // вставляем по текущему итератору
    ++it; // сдвигаем итератор после вставки
}

// Записываем обновлённый вектор обратно в файл
fstream output(filename, ios::out | ios::trunc);
if (!output) return -1;

```

```

    for (const auto& record : records) {
        output << record;
    }
    output.close();

    return countToAdd;
}

```

```
#endif
```

Main.cpp

```

#include "file_operations.h"
#include <limits>

```

```
using namespace std;
```

```
// Очистка буфера ввода после ошибок чтения
```

```

void clearInput() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

```

```
// Точка входа в программу. Реализует консольное меню для управления файлом.
```

```

int main() {
    // Установка русской локали для корректного отображения текста
    setlocale(LC_ALL, "ru");

    char filename[100];           // буфер для имени файла
    Pair searchValue;             // значение для поиска/сравнения
    double decrementValue;       // величина уменьшения (параметр L)
    int recordsCount;            // количество записей (параметр K)
    int choice;                  // выбор пункта меню

    // Главный цикл программы
    do {
        // Вывод меню

```

```

cout << "\n===== МЕНЮ ====="

<< "\n1. Создать файл с записями"
<< "\n2. Показать содержимое файла"
<< "\n3. Удалить записи (равные заданному)"
<< "\n4. Уменьшить записи на значение L"
<< "\n5. Добавить записи после указанной"
<< "\n0. Выход"
<< "\n===== "
<< "\nВыберите действие: ";

// Проверка корректности ввода числа
if (!(cin >> choice)) {
    clearInput();
    cout << "Ошибка ввода! Попробуйте снова.\n";
    continue;
}

// Обработка выбора пользователя
switch (choice) {
case 1: // Создание файла
    cout << "Имя файла: ";
    cin >> filename;
    if (createFile(filename) < 0)
        cout << "Не удалось создать файл!\n";
    else
        cout << "Файл успешно создан\n";
    break;

case 2: // Просмотр файла
    cout << "Имя файла: ";
    cin >> filename;
    if (displayFile(filename) < 0)
        cout << "Не удалось прочитать файл!\n";
    break;

case 3: // Удаление записей
    cout << "Имя файла: ";
    cin >> filename;

```

```

cout << "Значение для удаления (целое вещественное): ";
cin >> searchValue;
{
    int result = deleteMatchingRecords(filename, searchValue);
    if (result < 0)
        cout << "Ошибка при удалении!\n";
    else
        cout << "Удалено записей: " << result << "\n";
}
break;

case 4: // Уменьшение записей на L
    cout << "Имя файла: ";
    cin >> filename;
    cout << "Значение для поиска (целое вещественное): ";
    cin >> searchValue;
    cout << "На сколько уменьшить (L): ";
    cin >> decrementValue;
    {
        int result = decreaseMatchingRecords(filename, searchValue,
decrementValue);
        if (result < 0)
            cout << "Ошибка при изменении!\n";
        else
            cout << "Изменено записей: " << result << "\n";
    }
    break;

case 5: // Добавление записей после указанной
    cout << "Имя файла: ";
    cin >> filename;
    cout << "Значение-ориентир (целое вещественное): ";
    cin >> searchValue;
    cout << "Сколько записей добавить: ";
    cin >> recordsCount;
    {
        int result = insertRecordsAfter(filename, searchValue, recordsCount);
        if (result < 0)

```

```
        cout << "Ошибка при добавлении!\n";
    else
        cout << "Добавлено записей: " << result << "\n";
    }
    break;

case 0: // Завершение программы
    cout << "Программа завершена. До свидания!\n";
    break;

default: // Неверный пункт меню
    cout << "Неверный пункт меню! Выберите 0-5.\n";
}
} while (choice != 0);

return 0;
}
```

Результаты работы:

```
===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
=====
Выберите действие: 1
Имя файла: test
Enter the number of entries: 3
Record #1 (integer real): 12
13
Record #2 (integer real): 17
68
Record #3 (integer real): 34
98
Файл успешно создан

===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
=====
Выберите действие: 2
Имя файла: test

--- File Contents ---
12 : 13
17 : 68
34 : 98
--- Total records: 3 ---

===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
=====
Выберите действие: 3
Имя файла: test
Значение для удаления (целое вещественное): 12
13
Удалено записей: 1
```

```
=====
Выберите действие: 4
Имя файла: test
Значение для поиска (целое вещественное): 17
68
На сколько уменьшить (L): 10
Изменено записей: 1
```

```
===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
```

```
=====
Выберите действие: 2
Имя файла: test
```

```
--- File Contents ---
75 : 0
84 : 98
--- Total records: 2 ---
```

```
===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
```

```
=====
Выберите действие: 5
Имя файла: test
Значение-ориентир (целое вещественное): 34
98
Сколько записей добавить: 2
New entry #1 (integer real): 10
84
New entry #2 (integer real): 76
82
Добавлено записей: 2
```

```
===== МЕНЮ =====
1. Создать файл с записями
2. Показать содержимое файла
3. Удалить записи (равные заданному)
4. Уменьшить записи на значение L
5. Добавить записи после указанной
0. Выход
```

```
=====
Выберите действие: 2
Имя файла: test
```

```
--- File Contents ---
75 : 0
84 : 98
10 : 34
76 : 32
--- Total records: 4 ---
```

UML:

Pair
- first: int - second: double
+ Pair() + Pair(f: int, s: double) + getFirst(): int + getSecond(): double + operator<(other: Pair): bool + operator>(other: Pair): bool + operator==(other: Pair): bool + operator!=(other: Pair): bool + operator--(): Pair& + operator--(int): Pair + subtract(value: double): void <<friend>> + operator<<(ostream&, Pair): ostream& + operator>>(istream&, Pair): istream& + operator<<(fstream&, Pair): fstream& + operator>>(fstream&, Pair): fstream&

Вопросы:

1. Поток – последовательность байтов, не зависящая от конкретного устройства (оперативная память, файл, клавиатура, принтер). Обмен с потоком обычно производится через буфер.
2. Потоки бывают стандартные (однонаправленные), строковые (одно- и двунаправленные), файловые (одно- и двунаправленные).
3. #include <iostream>
4. #include <fstream>
5. #include <sstream>
6. Операция сдвига влево operator<<.
7. Операция сдвига вправо operator>>.

8. `put(char c)`, `write(const char* buf, int size)`, а также оператор `<<`.
9. `get()`, `get(char &c)`, `read(char* buf, int size)`, `getline()`, а также оператор `>>`.
10. Режимы: `in`, `out`, `trunc`, `app`, `ate`, `binary`.
11. `ios::app`.
12. `ifstream file("f.txt")` – режим `in` (открыть для чтения, по умолчанию для `ifstream`).
13. `fstream file("f.txt")` – режим `in|out` (чтение и запись, файл должен существовать, если не указан `trunc`).
14. `ofstream file("f.txt")` – режим `out|trunc` (стирание и запись, если файла нет – создаётся).
15. Открытие для дозаписи в конец файла (если файла нет, он создаётся).
16. Открытие для записи с удалением старого содержимого (стирание и запись).
17. Открытие для чтения и записи с удалением старого содержимого (стирание, чтение, запись; если файла нет – создаётся).
18. Используя режим `in` (например, `ifstream stream("file.txt")` или `fstream stream("file.txt", ios::in)`).
19. Используя режим `out` (например, `ofstream stream("file.txt")` или `fstream stream("file.txt", ios::out)`).
20. Примеры: `ofstream out("file.txt");` `ofstream out("file.txt", ios::app);` `ifstream in("file.txt");` `fstream io("file.txt", ios::in|ios::out);` `fstream io("file.txt", ios::in|ios::out|ios::trunc);`
21. Чтение объектов из потока: `stream >> obj;` `stream.get(c);` `stream.read(buf, size);` `stream.getline(str, count);` `while(stream >> obj) { ... }`
22. Запись объектов в поток: `stream << obj;` `stream.put(c);` `stream.write(buf, size);`
23. Алгоритм удаления записей: открыть исходный файл для чтения, открыть временный файл для записи, читать объекты, если номер (или условие) не совпадает с удаляемым – записывать во временный, закрыть оба файла, удалить исходный, переименовать временный в исходный.
24. Алгоритм добавления записей: для добавления в конец – открыть файл в режиме `app` и записать. Для добавления в середину/начало – использовать вспомогательный файл: открыть исходный для чтения, временный для записи, при достижении позиции вставки записать новый объект, затем

скопировать остальные, закрыть, удалить исходный, переименовать временный.

25. Алгоритм изменения записей: открыть исходный файл для чтения, временный для записи, читать объекты, если номер записи совпадает с изменяемым – спросить подтверждение, записать изменённый объект, иначе записать старый, закрыть, удалить исходный, переименовать временный.